Guia QA

Setor de Inteligência Artificial

Versão 3.0 - Abril 2026

1. Para quem é este guia

Este guia é para o QA que vai operar o BKPilot no dia a dia. Ele foi escrito para quem está começando no projeto e precisa entender não só "qual comando rodar", mas também:

- o que o agente realmente faz;
- o que continua sendo responsabilidade do QA humano;
- como escolher o fluxo certo;
- como interpretar os artefatos gerados;
- como evitar erros operacionais comuns.

Você não precisa ser desenvolvedor para usar o agente com segurança. Mas precisa entender o processo, porque o agente não substitui julgamento de QA. Ele automatiza a execução, a coleta de evidências e a consolidação dos resultados; a qualidade final do trabalho ainda depende da sua condução.

2. O que é o BKPilot

O BKPilot é uma plataforma local de automação de QA orientada a skills. Em termos práticos, ele transforma atividades manuais e repetitivas de teste em fluxos executáveis via CLI.

Em vez de abrir navegador, navegar tela por tela, registrar erro manualmente, capturar screenshot, montar planilha, escrever bug card e gerar relatório na mão, o QA opera esse fluxo por meio de comandos padronizados.

O agente combina quatro coisas:

- skills declarativas: os comandos de QA como `/explorar`, `/testar-modulo` e `/gerar-relatorio`;
- automação de browser: Playwright para navegar, clicar, preencher e capturar evidências;
- engine operacional: scripts e módulos Node.js para lote, reteste, geração de PDF e tratamento de arquivos;
- estrutura multi-tenant: tudo que é específico de um cliente fica isolado no pack desse cliente.

3. O que ele não é

É importante não criar expectativa errada.

O agente:

- não substitui análise crítica do QA;
- não valida sozinho se um comportamento faz sentido de negócio;
- não decide por conta própria o que é aceitável para o cliente;
- não elimina necessidade de revisão de cenário, reteste e fechamento técnico;
- não é um portal web onde o QA trabalha diariamente.

A operação do QA é local, via CLI. A plataforma web do ecossistema BKPilot é um portal de gestão e entrega, não o ambiente principal de execução.

4. Como o QA opera o agente hoje

Hoje o BKPilot pode ser operado via múltiplas CLIs de agente:

- Claude Code
- Codex
- OpenCode

Isso é importante porque o projeto não é exclusivo de uma única ferramenta.

4.1 Diferença entre CLI e skill

A skill é o comportamento de QA.

A CLI é o ambiente que executa esse comportamento.

Exemplo:

- `/explorar` é a skill;
- Claude Code, Codex ou OpenCode são ambientes possíveis para rodar essa skill.

Ou seja: a BKPilot não depende conceitualmente de uma única CLI. A fonte canônica das skills é uma só, e o projeto gera distribuições para múltiplos targets.

4.2 Regra prática para o QA

Se o time te orientar a usar uma CLI específica, siga o padrão do projeto ou da squad.

Se houver liberdade operacional:

- use Claude Code quando a operação exigir maior maturidade no fluxo já consolidado do time;
- use Codex quando estiver operando no target preparado para Codex;
- use OpenCode quando o time estiver trabalhando nesse front-end ou usando provedores ligados a ele.

O que não muda entre as CLIs é o processo de QA. O que muda é o ambiente operacional.

5. Como a arquitetura impacta seu trabalho

Você não precisa decorar a arquitetura inteira, mas precisa entender três pilares:

5.1 Fonte única das skills

As skills são mantidas de forma centralizada e distribuídas para diferentes CLIs. Isso significa que o comportamento esperado do agente não deve depender de uma edição manual isolada em uma CLI específica.

5.2 Isolamento por cliente

Cada cliente tem seu pack próprio em `clients/<id>/`. Nesse pack ficam:

- URL base;
- login do cliente;
- flows especializados;
- bugs exportados;
- fixtures;
- cenários e materiais do cliente.

Para o QA, isso significa uma coisa simples: quando você entra em um cliente novo, a referência correta é o pack daquele cliente, não arquivos soltos espalhados.

5.3 Evidência e observabilidade não são opcionais

Toda execução séria de browser precisa deixar rastro:

- screenshot;
- vídeo;
- log de console;
- log de rede;
- log de cleanup;
- progresso da execução.

Se não existe evidência em disco, você não deve considerar o item como validado.

6. O que o agente faz automaticamente

Dependendo da skill, o agente pode fazer automaticamente:

- abrir e controlar o browser;
- autenticar com o usuário de QA;
- navegar por páginas e fluxos;
- capturar screenshots;
- gravar vídeo;
- monitorar `console.error` e `console.warning`;

- registrar falhas e lentidão de rede;
- reautenticar quando a sessão expira;
- registrar e limpar dados de teste;
- atualizar planilhas e artefatos;
- transformar falhas em bug cards;
- consolidar relatórios parciais e finais.

Isso reduz esforço operacional, mas não elimina sua responsabilidade de revisar o resultado.

7. Pré-requisitos

Antes da primeira operação, confirme estes itens:

- Node.js 18+ instalado;
- dependências do projeto instaladas;
- browsers do Playwright instalados;
- `clients/<id>/.env` configurado com a senha de QA do cliente alvo;
- CLI operacional do time disponível;
- acesso ao ambiente do cliente;
- certeza de que o ambiente é de dev/homologação quando houver ação mutativa.

7.1 Sobre a senha de QA

A senha nunca deve ser passada inline no comando.

Errado:

```
--login email@cliente.com:senha
```

Certo:

```
--login email@cliente.com
```

Com a senha vindo de `clients/<id>/.env` (arquitetura multi-tenant: cada cliente tem seu próprio `.env` isolado).

Essa regra existe por segurança. Senha inline vaza para histórico de terminal, logs e captura de comando.

8. Primeira configuração

8.1 Clonar o repositório

```
 git clone <URL-DO-REPOSITORIO> bugkillers-qa-agent
cd bugkillers-qa-agent
```

8.2 Rodar o setup

```
bash setup.sh
```

O `setup.sh` existe para reduzir erro humano na primeira instalação. Em vez de depender de instalação manual fragmentada, ele:

- verifica pré-requisitos básicos;
- prepara o ambiente;
- ajusta a configuração necessária;
- cria a estrutura inicial esperada pelo projeto.

8.3 Configurar credenciais (multi-tenant)

O `setup.sh` é executado **uma vez por máquina** — não cria credenciais de cliente. Para cada cliente novo, use o script dedicado:

```
./novo-cliente.sh <id> --nome "Nome do Cliente" --url https://app.cliente.com.br
```

Isso cria `clients/<id>/` com `.env`, `config.json` e `login.js` skeleton. Depois:

1. **`clients/<id>/.env`** — preencha:

   ```
   QA_PASSWORD=sua_senha_aqui
   ```

2. **`clients/<id>/config.json`** — ajuste `postLoginSelector` (selector que aparece após login bem-sucedido) e `defaultFlow` / `defaultRetesteFlow` se aplicável.

3. `clients/<id>/login.js` — ajuste os seletores de usuário/senha conforme a tela de login real.

O `.env` da raiz do projeto **não armazena** `QA_PASSWORD` — ele é exclusivo de integrações globais (Jira, GitHub Issues). A senha de QA mora sempre em `clients/<id>/.env`.

Se o cliente exigir variável específica, siga o contrato configurado em `envPassword` do `config.json` do client pack.

9. Playwright: o que você realmente precisa entender

O projeto usa Playwright como camada de automação de browser.

Para o QA, o ponto mais importante não é a implementação técnica interna, e sim o efeito prático:

- o agente consegue interagir com a aplicação real;
- a execução pode produzir evidência visual;
- erros de frontend e rede podem ser capturados junto com o teste.

Você não precisa escrever script Playwright para operar o fluxo normal do BKPilot. O Playwright está embutido na operação das skills.

10. O pipeline principal de trabalho

Na maior parte dos projetos, o trabalho segue esta lógica:

1. entender o sistema;
2. gerar cenários;
3. executar testes;
4. consolidar bugs;
5. gerar entregáveis.

No agente isso normalmente vira:

```
/explorar -> /gerar-cenarios -> /testar-modulo ou /executar-planilha
```

```
-> /relatorio-parcial -> /reportar-bug -> /gerar-relatorio
```

Quando o ciclo inclui automação de testes para o cliente, o ponto de partida é `/plano-automacao`, que decide se o sistema está pronto para receber investimento em automação antes de acionar `/gerar-automacao-cliente`.

Agora o ponto importante: isso não é uma sequência decorativa. Cada etapa prepara a próxima.

10.1 `/explorar`

Objetivo

Mapear o sistema antes da execução formal. Essa skill existe para identificar telas, fluxos, elementos, endpoints e sinais técnicos do comportamento da aplicação.

Quando usar

Use no início do projeto, quando entra em um módulo novo, quando a aplicação mudou de forma relevante ou quando você percebe que está executando sem contexto suficiente.

Sem uma exploração boa, a geração de cenários e a execução seguinte ficam mais frágeis.

Exemplo

```
/explorar https://app.cliente.com.br --login qa@cliente.com
```

Cuidados

- não trate `/explorar` como burocracia;
- valide se os artefatos realmente foram gerados;
- se a exploração vier incompleta, a planilha pode ficar bonita, mas continuar errada na base.

10.2 `/gerar-cenarios`

Objetivo

Transformar o entendimento do sistema em cenários estruturados e executáveis.

Quando usar

Use depois da exploração, quando já existe base suficiente para converter conhecimento do sistema em planilha de teste.

Mesmo com geração automática, a planilha continua exigindo revisão humana.

Exemplo

```
/gerar-cenarios --formato gherkin
```

Exemplo com foco de módulo

```
/gerar-cenarios --formato gherkin --modulo Login
```

Cuidados

- revise cobertura de áreas críticas;
- procure passos vagos ou duplicados;
- confirme se a planilha reflete o escopo real do ciclo.

10.3 `/testar-modulo`

Objetivo

Executar testes de forma organizada por módulo funcional, combinando roteiro e exploração complementar.

Quando usar

Use quando o projeto estiver segmentado por áreas funcionais e você quiser melhor rastreabilidade de cobertura, bugs e evidências por domínio.

É especialmente útil em divisões como Login, Cadastro, Pedidos, Relatórios e Administração.

Exemplo

```
/testar-modulo Login clients/tega/cenarios/cenarios.xlsx --login adminteste2
```

Cuidados

- não trate a skill como mero executor de CTs;
- use-a quando quiser também descobrir falhas que a planilha sozinha não cobriria;
- confira se as evidências do módulo foram realmente salvas.

10.4 `/executar-planilha`

Objetivo

Executar a planilha em lote com mais controle operacional.

Quando usar

Use quando a planilha já está madura e o foco principal é throughput, reexecução ou batch controlado.

É especialmente útil em reexecução em massa, filtro por prioridade, retries e circuit breaker.

Exemplo

```
/executar-planilha clients/tega/cenarios/cenarios.xlsx --login adminteste2
```

Exemplo com filtro

```
/executar-planilha clients/tega/cenarios/cenarios.xlsx --prioridade Alta --login admi
```

Exemplo com retry

```
/executar-planilha clients/tega/cenarios/cenarios.xlsx --login adminteste2 --retry 2
```

Cuidados

- use batch quando a planilha já estiver suficientemente estável;
- se a base estiver ruim, você só acelera erro em escala;
- revise retries e limites para não mascarar falha estrutural.

10.5 `/reportar-bug`

Objetivo

Transformar falhas consolidadas em bug cards utilizáveis.

Quando usar

Use depois da execução, quando já existem artefatos suficientes para estruturar bugs de forma defensável.

O valor aqui não é só descrever erro; é consolidar contexto, evidência, esperado, observado, severidade e impacto.

Exemplo

```
/reportar-bug --fonte resultado/latest/
```

Exemplo usando base anterior

```
/reportar-bug --fonte resultado/latest/ --bugs-anteriores clients/tega/clients/<id>/r
```

Cuidados

- bug card bom não é o mais longo;
- bug card bom é o mais reproduzível e mais defensável;
- sem boa evidência, a skill não compensa material ruim de origem.

10.6 `/gerar-relatorio`

Objetivo

Consolidar o ciclo em um entregável formal.

Quando usar

Use quando o trabalho operacional já precisa virar material consumível por liderança, cliente ou gestão.

Exemplo

```
/gerar-relatorio --cliente "TEGA Sistemas" --formato pdf
```

Exemplo alternativo

```
/gerar-relatorio --cliente "TEGA Sistemas" --formato docx
```

Cuidados

- não use essa skill como atalho para encobrir execução mal consolidada;
- relatório bom depende de execução boa, bugs bons e evidência íntegra.

10.7 `/relatorio-parcial`

Objetivo

Gerar visibilidade intermediária durante o ciclo.

Quando usar

Use em projetos longos, acompanhamento semanal ou momentos em que o cliente precisa de visão parcial do andamento.

Exemplo

```
/relatorio-parcial --cliente "TEGA Sistemas" --semana 1
```

Exemplo com observações

```
/relatorio-parcial --cliente "TEGA Sistemas" --semana 1 --notas "Ambiente instável na
```

Cuidados

- parcial não é relatório final resumido;
- ela existe para dar clareza de andamento, risco e bloqueios no meio do ciclo.

10.8 `/plano-automacao`

Objetivo

Avaliar a viabilidade técnica e o retorno de investimento da automação de testes antes de qualquer proposta de entrega de código ao cliente.

Quando usar

Use no início de projetos que envolvam automação de testes, quando o cliente solicita uma proposta de automação ou quando você precisa decidir se o sistema está maduro o suficiente para justificar o investimento.

Essa skill é a porta de entrada do pipeline de automação. Ela pode economizar semanas de trabalho em um sistema que ainda não está pronto para automação.

Exemplo

```
/plano-automacao https://app.cliente.com.br --login qa@cliente.com --dominio "ERP de
```

Cuidados

- não ignore o veredito da skill; se o sistema for classificado como "Não Viável", a automação provavelmente será frustrante;
- revise o `scores_automatizabilidade.json` gerado; ele é o gate check obrigatório da skill;
- todo relatório `.md` destinado ao cliente (plano, estimativa, viabilidade) deve ter `.pdf` correspondente.

Guia dedicado:

```
documentacao_projeto/Guia QA - Skill plano-automacao.md
```

10.9 Skills de automação do cliente

As skills `/gerar-automacao-cliente` e `/auditar-automacao-cliente` possuem guia próprio porque geram pacote de código para entrega ao cliente.

Use o guia dedicado antes de operar esse fluxo:

```
documentacao_projeto/Guia QA - Skills de Automacao do Cliente.md
```

Regras principais desse fluxo:

- o pacote do cliente fica em `clients/<id>/entregaveis/automacao/<stack>/`;
- todo relatório `.md` destinado ao cliente deve ter `.pdf` correspondente;
- a auditoria deve corrigir defeitos técnicos objetivos antes da entrega;
- arquivos de governança em `clients/<id>/clients/<id>/resultado/<timestamp>/governanca/` são internos e não devem ser enviados ao cliente.

11. Quando usar os comandos avulsos

Nem todo trabalho segue o pipeline completo. Algumas demandas são pontuais.

11.1 `/executar-fluxo`

Objetivo

Executar um fluxo E2E específico sem depender do batch maior.

Quando usar

Use quando o time precisa validar uma jornada muito específica, como login -> dashboard -> criar pedido -> checkout.

Exemplo

```
/executar-fluxo "login -> dashboard -> criar pedido -> checkout" --login qa@cliente.c
```

Exemplo com dados

```
/executar-fluxo "criar pedido com dados" --login qa@cliente.com --dados dados.json
```

Cuidados

- não use fluxo isolado para fingir cobertura ampla;
- ele é ótimo para validação dirigida, não para substituir ciclo completo.

11.2 `/testar-forms`

Objetivo

Fazer uma bateria profunda sobre formulários.

Quando usar

Use quando a qualidade do formulário é a prioridade principal, não a cobertura ampla do sistema.

É especialmente indicado para validação pesada, máscara, upload, limites, internacionalização e cenários negativos.

Exemplo

```
/testar-forms https://app.cliente.com.br/cadastro --login qa@cliente.com
```

Cuidados

- formulários críticos merecem profundidade própria;
- não assuma que a execução geral vai capturar tudo que um form complexo pode quebrar.

11.3 `/regressao`

Essa é uma das skills mais densas do projeto e merece ser entendida com mais cuidado.

Ela não serve apenas para "retestar bug corrigido". Na prática, a `/regressao` opera em três modos diferentes:

- Modo Planilha: retesta cenários que já falharam em uma execução anterior;
- Modo Bugs: retesta bugs vindos de uma pasta de export do Jira ou estrutura manual;
- Modo Combinado: retesta os bugs e ainda cruza esses bugs com a planilha original para validar cenários relacionados.

Isso significa que a skill cobre dois trabalhos que o QA costuma separar mentalmente, mas que no produto estão conectados:

- reteste do defeito reportado;
- checagem de regressão ao redor da correção.

Em termos práticos, use `/regressao` quando já houve correção e o objetivo agora é confirmar:

- se o bug foi corrigido;
- se o comportamento anterior realmente não persiste;
- se a correção introduziu efeito colateral;
- se cenários próximos ou relacionados continuam íntegros;
- se a nova evidência é comparável à evidência anterior.

Quando usar cada modo

Use o Modo Planilha quando seu foco for reexecutar cenários conhecidos e falhos de um ciclo anterior, de forma mais previsível.

Use o Modo Bugs quando você recebeu um pacote de bugs do cliente, do Jira ou de uma entrega anterior, e precisa validar item por item sem depender de uma planilha de contexto.

Use o Modo Combinado quando quer fazer o trabalho mais completo. Esse é o modo mais forte da skill, porque ele não fica preso ao bug isolado: ele usa a planilha como contexto para buscar cenários relacionados e ampliar a segurança da validação.

Por que essa skill é mais crítica que parece

Muita gente trata regressão como "abrir a tela e ver se o erro sumiu". Essa skill vai além disso.

Ela foi desenhada para:

- ingerir bugs externos em formatos reais de operação, como Excel, Word ou XML do Jira;
- reaproveitar evidências anteriores;
- comparar antes e depois;
- classificar o resultado como `Corrigido`, `Persiste`, `Regressão` ou `Inconclusivo`;
- cruzar bugs com cenários da planilha original;
- ajudar a detectar regressão sistêmica, não apenas regressão pontual.

Em outras palavras: `/regressao` não é só uma skill de confirmação. Ela é também uma skill de defesa de release.

O que o QA precisa observar ao usar `/regressao`

Por ser uma skill densa, ela exige mais disciplina do operador.

O QA deve prestar atenção em:

- qualidade dos dados de entrada, especialmente pasta de bugs e planilha;
- existência de evidência anterior para comparação;
- classificação correta do resultado final;
- presença de screenshot obrigatório por bug retestado;
- diferença entre "bug corrigido" e "sistema segue saudável ao redor da correção".

Se o reteste validar apenas o bug isolado, mas os cenários vizinhos quebrarem, o fechamento correto não é simplesmente Corrigido. É justamente aí que o valor real da skill aparece.

Exemplos práticos de uso

Exemplo 1: Modo Planilha

Use quando você quer reexecutar cenários que já falharam antes, sem depender de uma pasta de bugs externa.

```
/regressao --planilha clients/tega/cenarios/cenarios.xlsx --login adminteste2
```

Esse modo é útil quando:

- o ciclo anterior já deixou a planilha marcada com falhas;
- o time quer validar novamente os mesmos cenários;
- o foco é estabilidade de execução e não ingestão de bugs vindos do Jira.

Exemplo 2: Modo Bugs

Use quando o material principal de entrada é uma pasta de bugs exportada do Jira.

```
/regressao --client tega --bugs clients/tega/bugs/2026-04-17/ --login adminteste2
```

Esse modo é útil quando:

- o cliente ou a squad devolveu um lote de bugs como corrigidos;
- você precisa validar bug por bug;

- ainda não quer ou não consegue usar a planilha como contexto adicional.

Exemplo 3: Modo Combinado

Use quando você quer o cenário mais completo e mais seguro.

```
/regressao --client tega --bugs clients/tega/bugs/2026-04-17/ --planilha clients/tega/
```

Esse modo é útil quando:

- você quer retestar os bugs corrigidos;
- quer cruzar esses bugs com cenários relacionados;
- quer reduzir a chance de aprovar uma correção pontual que quebrou outra área do sistema.

Exemplo 4: Com visual diff

Use quando há valor em comparar visualmente a evidência antiga com a nova.

```
/regressao --client tega --bugs clients/tega/bugs/2026-04-17/ --planilha clients/tega/
```

Esse uso é especialmente útil em:

- bugs de layout;
- regressão visual;
- problemas de modal, botão, tabela ou editor gráfico;
- situações em que o parece corrigido ainda precisa de comparação objetiva.

Como escolher entre Planilha, Bugs e Combinado

Se a sua dúvida for "qual modo devo usar?", pense da seguinte forma:

Use Planilha quando:

- sua referência principal é a execução anterior dos cenários;
- você está revalidando CTs falhos;
- o lote de bugs externo não é o centro da operação.

Use Bugs quando:

- a entrada mais importante é uma exportação do Jira;
- o trabalho está orientado a bug card;

- você precisa de reteste pontual sem depender de contexto amplo.

Use Combinado quando:

- você quer a validação mais robusta;
- o risco da correção é alto;
- o módulo é sensível;
- você precisa defender melhor a decisão de fechar como corrigido.

Em geral, se houver tempo e material suficiente, o Modo Combinado tende a ser a melhor escolha, porque ele reduz visão curta de reteste.

11.4 `/acessibilidade`

Objetivo

Executar auditoria focada em acessibilidade.

Quando usar

Use quando a exigência principal é conformidade ou diagnóstico de acessibilidade, sem depender de um ciclo funcional completo.

Exemplo

```
/acessibilidade https://app.cliente.com.br --login qa@cliente.com --nivel AA
```

Cuidados

- acessibilidade não deve ser tratada como detalhe opcional de front-end;
- use essa skill para enxergar problemas que o teste funcional puro não mostra.

11.5 `/performance`

Objetivo

Avaliar desempenho e experiência de carregamento.

Quando usar

Use quando houver suspeita de lentidão, comparação entre ambientes ou necessidade de olhar Core Web Vitals e percepção de carga.

Exemplo

```
/performance https://app.cliente.com.br --login qa@cliente.com --throttle 4g
```

Cuidados

- problema de performance nem sempre aparece como erro explícito;
- a skill ajuda a qualificar lentidão que o usuário sente, mas o log sozinho não explica.

11.6 `/api-check`

Objetivo

Analisar a camada de API com foco técnico.

Quando usar

Use quando auth, erros, payloads, contratos e robustez da API forem parte importante da investigação.

Exemplo

```
/api-check https://app.cliente.com.br --login qa@cliente.com --endpoints auto
```

Cuidados

- use essa skill quando o problema parece vir do backend, não só da interface;
- ela é especialmente útil quando o bug funcional está sendo causado por API frágil.

11.7 `/usabilidade`

Objetivo

Avaliar usabilidade de forma heurística.

Quando usar

Use quando o problema principal não é um erro técnico estrito, mas fricção de uso, inconsistência de navegação ou decisão ruim de interface.

Exemplo

```
/usabilidade https://app.cliente.com.br --login qa@cliente.com --fluxos "login -> dasl
```

Cuidados

- nem todo problema relevante gera stacktrace;
- essa skill ajuda a defender problema de experiência com linguagem de avaliação, não só de erro.

11.8 `/testar-ia`

Objetivo

Validar fluxos ligados a assistentes e comportamento de IA.

Quando usar

Use quando o ciclo envolve chat, automação baseada em IA, robustez, guardrails, permissões, comportamento da resposta ou segurança do assistente.

Exemplo

```
/testar-ia --client tega --login adminteste2
```

Cuidados

- teste de IA não é só perguntar qualquer coisa e ver se responde;
- é preciso olhar robustez, alucinação, escopo, segurança e consistência de comportamento.

11.9 `/gerar-relatorio` e `/relatorio-parcial` como skills de entrega

Mesmo já aparecendo no pipeline principal, vale reforçar: essas duas skills também são usadas como ferramentas de comunicação e não só como

fechamento burocrático.

Use `/relatorio-parcial` quando a prioridade é transparência durante o ciclo.

Use `/gerar-relatorio` quando a prioridade é fechamento formal e entrega consolidada.

11.10 `/push-bugs`

Objetivo

Empurrar bug cards consolidados para o sistema de acompanhamento do time.

Quando usar

Use quando o ciclo já produziu bugs bons e o próximo passo é registrar esse material em Jira ou fluxo equivalente.

Exemplo

```
/push-bugs --fonte resultado/latest/
```

Cuidados

- só faz sentido usar `/push-bugs` quando a base já está boa;
- publicar bug ruim mais rápido não melhora o processo;
- revisão continua sendo necessária antes da publicação.

12. Artefatos: como ler o que foi gerado

Um QA que não entende os artefatos gerados perde metade do valor do agente.

12.1 `clients/<id>/estado/`

Representa o conhecimento persistido sobre o sistema explorado daquele cliente.

Normalmente inclui:

- `mapa.md`

- `fluxos.md`
- `elementos.json`
- `api_endpoints.json`

Pense nessa pasta como memória estruturada da exploração.

12.2 `clients/<id>/resultado/<timestamp>/`

É a pasta da execução.

Ali ficam:

- screenshots;
- vídeos;
- `console_log.json`;
- `network_log.json`;
- `cleanup_log.json`;
- `progress.log`;
- summaries e saídas derivadas.

Se você quer entender "o que aconteceu naquela run", é aqui que deve olhar primeiro.

12.3 `clients/<client>/cenarios/`

Contém materiais persistentes do cliente:

- planilhas;
- fichas de risco;
- perfis;
- arquivos auxiliares do ciclo.

12.4 `clients/<client>/bugs/<data>/`

Contém exports, anexos e material de reteste ligado a bugs daquele cliente.

13. Regras críticas de operação

13.1 Evidência visual é obrigatória

Se uma validação de browser foi feita, deve existir prova em disco.

Sem evidência:

- não feche como validado;
- não trate como concluído;
- não use linguagem absoluta no relatório.

13.2 Cleanup não é detalhe

Se o teste cria dado, o ciclo precisa considerar limpeza.

Quando a limpeza não for possível:

- registre no `cleanup_log.json`;
- deixe explícito o que ficou pendente;
- não finja reversão que não aconteceu.

13.3 Reautenticação não deve mascarar problema real

Se a sessão expira e o agente se recupera, ótimo. Mas isso não significa que a instabilidade deixa de ser relevante. O QA ainda precisa avaliar se a recorrência desse comportamento merece menção.

13.4 Ambiente importa

Nunca trate ação mutativa como trivial. Se o projeto pode escrever no sistema, garanta que está em dev ou homologação.

14. Problemas comuns e como pensar sobre eles

14.1 Falta de credencial

Se o `clients/<id>/.env` está errado, quase todo o fluxo quebra cedo.

Sinal típico:

- login falha;
- execução trava no início;
- reauth entra em loop;

- comandos reclamam de variável ausente.

Primeiro diagnóstico: revisar `clients/<id>/.env` e o contrato de `envPassword` no `config.json` do cliente.

14.2 Erro de login

Nem sempre o problema é a senha. Pode ser:

- seletor de login desatualizado;
- mudança na tela;
- captcha;
- MFA;
- problema do ambiente.

A postura correta é separar erro de credencial de erro de fluxo de autenticação.

14.3 Artefato ausente

Se faltou screenshot, vídeo ou log:

- não assuma que "deve ter sido gerado";
- confirme a pasta correta;
- valide o timestamp;
- confira se a execução foi realmente finalizada.

14.4 Exploração ruim gera efeito cascata

Se o `/explorar` saiu incompleto, a planilha gerada depois pode parecer organizada, mas ainda assim estar errada na base.

Sempre desconfie de boa forma com base ruim.

15. Checklist operacional do QA

Antes de iniciar:

- [ ] estou no cliente certo;
- [ ] revisei o pack em `clients/<id>/`;
- [ ] confirmei credenciais em `clients/<id>/.env`;

- ☐ sei se o ambiente aceita mutação;
- ☐ sei qual CLI vou usar;
- ☐ entendi o objetivo do ciclo.

Antes de fechar execução:

- ☐ os artefatos existem em disco;
- ☐ há logs de console e rede quando aplicável;
- ☐ as evidências relevantes estão salvas;
- ☐ pendências de cleanup foram registradas;
- ☐ o resultado faz sentido tecnicamente;
- ☐ o texto final não exagera conclusões sem prova.

16. Fluxo recomendado para quem está começando

Se for sua primeira operação real, siga esta sequência:

1. leia o pack do cliente em `clients/<id>/`;
2. confirme `clients/<id>/.env`;
3. rode a exploração;
4. revise o material gerado;
5. gere cenários;
6. revise a planilha;
7. execute um módulo pequeno primeiro;
8. confira os artefatos da run;
9. só depois avance para o restante do projeto.

Essa ordem existe para reduzir erro de escala. Começar pequeno é mais barato do que descobrir um problema estrutural após horas de execução.

17. Fechamento

O BKPilot não é só um conjunto de comandos. Ele é uma forma padronizada de operar QA com rastreabilidade, evidência e reutilização.

Quanto melhor você entender:

- o papel de cada skill;
- a arquitetura por cliente;
- a diferença entre execução e entrega;
- a importância dos artefatos;

melhor será a qualidade do seu trabalho com o agente.

Em caso de dúvida, não tente "compensar no improviso". Volte para:

- o pack do cliente;
- os artefatos da execução;
- a documentação do projeto;
- o fluxo padrão do pipeline.

BKPilot - Setor de Inteligência Artificial